



VISHWAKARMA
UNIVERSITY
Maximising Human Potential

Activity based

Project Report on

Artificial Intelligence

Project - I

Submitted to Vishwakarma University, Pune

Under the Initiative of

Contemporary Curriculum, Pedagogy, and Practice (C2P2)

By

Snehal Late

1) Problem Statement and Objectives:

The project aims to design algorithms that can effectively analyze the syntactic structure of Python code snippets and extract meaningful syntactic dependencies between tokens. This involves processing Python code from diverse sources, tokenizing it into meaningful units, and creating a system that can accurately represent and analyze Python syntax and structure.

Objectives:

1) Develop Efficient Code Parsing Algorithms

- The project will focus on creating algorithms that can read and break down Python code into its core components. These algorithms will analyze the structure of the code and understand the relationships between different elements, such as functions, variables, and expressions.

2) Design a Python-Specific Tokenization System

- Python has a unique syntax, and traditional tokenization methods may not always capture its structure accurately. The project aims to develop a tokenization system specifically designed for Python, ensuring that each piece of code is broken into meaningful units while following Python's rules.

3) Extract Dependencies Between Code Elements

- The system will not only break code into tokens but also analyze how different parts of the code depend on each other. For example, it will identify which variables are used inside a function, how different functions interact, and how expressions are structured.

4) Process Code from Various Sources

- To make the system robust, it will be designed to handle Python code from different sources such as open-source projects, online repositories (like GitHub), and programming forums. This ensures that the system can work with real-world code written by different programmers.

5) Ensure Accuracy in Representing Python's Syntax

- Python has several unique features, such as indentation-based blocks, dynamic typing, and list comprehensions. The project will ensure that the analysis accurately captures these characteristics so that the extracted dependencies are meaningful.

6)Build a Scalable Framework for Future Applications

- The system will be designed in a way that it can be extended for additional purposes, such as analyzing code quality, suggesting improvements (refactoring), or even helping students learn Python by identifying mistakes and providing insights into their code.

2) Overview of System/Problem:

The Python Syntactic Analysis System is designed to process and understand Python code by analyzing its structure, the relationships between its components, and how well it follows the rules of Python's syntax. This system will be useful for various applications, ranging from improving code quality to helping programmers learn Python more effectively.

Key Purposes of the System

1)Code Understanding:-

- One of the main goals of this system is to help users understand how different parts of Python code relate to each other. For example, it can explain how functions are defined, how variables are used, how loops and conditionals interact with each other, and where dependencies between different parts of the code lie. This level of understanding is essential for debugging, code optimization, and development.

2)Educational Tool:-

- The system can serve as a valuable resource for beginners learning Python. By analyzing the syntactic structure of Python code, the system can illustrate how Python syntax works, what common patterns are followed, and where mistakes or inconsistencies may arise. It can help new programmers better grasp the logic behind Python programs and the correct way to write code, making it easier to learn and apply best practices.

3)Code Quality Enhancement:-

- Through syntactic analysis, the system can identify areas of the code that may need improvement. It can suggest refactoring options or point out where code might violate best practices or common patterns, leading to more efficient and maintainable code. The system can also identify potential anti-patterns, such as unnecessarily complex functions or redundant code, and recommend simplifications.

System Workflow

1) Input Collection:

Gathers Python code from various sources (GitHub, tutorials, user projects) in different formats.

2) Preprocessing:

Cleans and standardizes code by removing whitespace, correcting indentation, and fixing inconsistencies.

3) Tokenization:

Breaks code into tokens (keywords, variables, operators, punctuation) for simplified analysis.

4) Syntactic Analysis:

Maps the structure of the code based on Python syntax rules, identifying functions, loops, and data flow.

5) Dependency Extraction:

Identifies relationships between variables, functions, and operations for deeper code understanding.

6) Output Generation:

Produces structured outputs like diagrams, summaries, or reports highlighting syntactic features and dependencies.

3) Role of Approach/Algorithm/method(NLP) in the application:

While Natural Language Processing (NLP) is typically used to analyze human languages, its techniques can also be effectively applied to programming languages like Python. In this project, NLP methods are adapted to help the system understand and analyze Python code in a structured and meaningful way. Here's how key NLP concepts contribute to the system:

1. Lexical Analysis (Tokenization)

Tokenization is the first step in processing code, where the code is broken down into smaller, meaningful units called tokens. In the context of NLP, tokenization is used to break down text into words or sentences. Similarly, in Python code, lexical analysis will divide the code into elements like:

- Keywords (e.g., `def`, `return`, `if`)
- Identifiers (e.g., variable names like `x`, `y`, `result`)
- Operators (e.g., `+`, `-`, `==`)
- Literals (e.g., numbers, strings like `"hello"`)
- Punctuation (e.g., commas, parentheses)
- However, Python's unique feature of indentation-based syntax needs to be taken into account when tokenizing. A specialized tokenizer will be used to handle this, ensuring that blocks of code are correctly interpreted in terms of their scope and structure.
- For example:

```
def add(a, b):
```

```
    result = a + b
```

```
    return result
```

2. Syntactic Parsing (AST Generation)

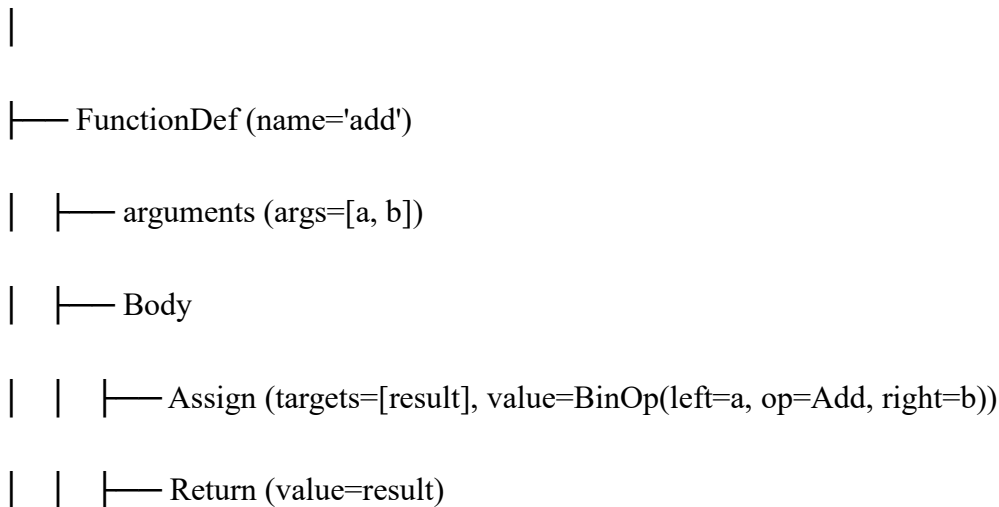
Once the code is tokenized, the next step is to parse it. In NLP, syntactic parsing involves analyzing the grammatical structure of a sentence, understanding how words fit together. Similarly, in programming languages like Python, syntactic parsing analyzes how tokens are arranged to follow the rules of Python syntax.

- The Abstract Syntax Tree (AST) is the core data structure used to represent the hierarchical structure of the code. It organizes code elements (such as functions, variables, and loops) in

a tree-like structure, where each node represents a code construct (e.g., a function definition, an assignment statement).

- The system will use context-free grammar parsing techniques to build this tree based on Python's grammar rules, ensuring the relationships between the tokens are correctly represented.
- For example:

Module



3. Dependency Parsing

Just as NLP uses dependency parsing to understand how words in a sentence depend on each other (e.g., subject-verb-object relationships), this system will use similar techniques to understand how elements of Python code depend on each other.

- For example:

Dependent Element	Head Element	Relationship
a, b	add (function)	Function parameters
result	= (assignment)	Assigned variable
a + b	= (assignment)	Right-hand side of assignment
+	a, b	Operator dependency
return	result	Return value dependency

- **Key NLP Algorithms and Methods Applied**

1) Finite-State Automata (FSA) for Tokenization

- Breaks Python code into keywords, identifiers, operators, literals, while respecting indentation and nested structures.
- Ensures correct handling of multi-character tokens (e.g., ==, +=) and punctuation.
- For example:

```
x = 10 + 5
```

```
if x == 15:
```

```
    print("Valid")
```

FSA States for Tokenization:

State	Input	Next State	Output Token
Start	x	Identifier State	IDENTIFIER(x)
Identifier	=	Operator State	OPERATOR(=)
Operator	1,0	Number State	NUMBER(10)
Number	+	Operator State	OPERATOR(+)
Operator	5	Number State	NUMBER(5)
Number	\n (newline)	Start	-
Start	if	Keyword State	KEYWORD(if)
Keyword	x	Identifier State	IDENTIFIER(x)
Identifier	==	Operator State	OPERATOR(==)
Operator	1,5	Number State	NUMBER(15)
Number	:	Punctuation State	PUNCTUATION(:)
Indent	print	Identifier State	IDENTIFIER(print)
Identifier	(	Punctuation State	PUNCTUATION((
String	"Valid"	String State	STRING("Valid")
String	)	Punctuation State	PUNCTUATION())

2) Recursive Descent Parsing for AST Construction

- Builds an Abstract Syntax Tree (AST) to represent the hierarchical structure of the code.

- Identifies function definitions, assignments, control structures, and operator usage.

For example:

FunctionDef(add)

├─ Arguments(a, b)

├─ Assign(result)

| └─ BinOp(+)

| └─ Name(a)

| └─ Name(b)

└─ Return(result)

3) Graph-Based Dependency Parsing for Syntactic Analysis

- Treats Python code as a graph, where nodes are tokens (functions, variables) and edges represent dependencies.
- Helps in code analysis, optimization, and refactoring suggestions by tracking function calls, assignments, and operations.

These techniques adapt NLP methods for structured Python code analysis, enabling better insights, static analysis, and transformation.

For example:

Input:

x = 10

y = x + 5


```
def add(a, b):
```

```
    return a + b
```

```
z = add(y, 2)
```

Dependency Graph Representation:

(10) (5) (2)

↓ ↓ ↓

[x] → (+) → [y] → [add()] → (+) → [z]

4) Data Representation/State Space Search:

Data Representation

The system will represent Python code in multiple layers:

1. **Raw Code Text:** The original source code
2. **Token Sequence:** Code broken down into meaningful tokens with metadata (type, position, etc.)
3. **Abstract Syntax Tree (AST):** Hierarchical representation of code structure
4. **Dependency Graph:** A directed graph representing relationships between code elements

State Space Search

Start State: Raw Python code snippets from various sources.

Goal State: A complete dependency graph and syntactic analysis of the code, showing relationships between all tokens and code blocks.

State Space: The set of all possible partial analyses of the code, from initial tokenization to complete dependency mapping.

Production Rules:

1. Tokenize the raw code according to Python lexical rules
2. Apply grammar rules to form valid AST nodes
3. Connect related tokens based on syntactic relationships
4. Identify and classify dependencies between code elements
5. Refine the syntactic structure through iterative analysis

5) Algorithm Design/Pseudocode:

- **Algorithm:**

Step 1: Tokenization Using tokenize Module

1. Initialize an empty list tokens to store tokenized results.
2. Define a set of punctuators ({, }, :, ,, etc.).
3. Read the input Python code using BytesIO.
4. Iterate through each token extracted by tokenize.tokenize():
 - If the token is an encoding marker, skip it.
 - If the token is a keyword, mark it as "Keyword".
 - If the token is a name (identifier), mark it as "Identifier".
 - If the token is a numeric literal, mark it as "Literal (Numeric)".
 - If the token is a string literal, mark it as "Literal (String)".
 - If the token is "True" or "False", mark it as "Literal (Boolean)".
 - If the token is in punctuators, mark it as "Punctuator".
 - If the token is an operator, mark it as "Operator".
 - Ignore newline and end-marker tokens.
 - Store the token and its type in tokens list.

5. Return the tokens list.

Step 2: Extract Syntactic Dependencies Using ast Module

1. Parse the input Python code into an Abstract Syntax Tree (AST).

2. Initialize an empty list dependencies to store syntax relationships.
3. Walk through the AST tree:
 - If the node is a function definition (FunctionDef), add a message indicating that the function is defined.
 - If the node is a function call (Call), extract the function name and add a message indicating that it is called.
 - If the node is a binary operation (BinOp), extract the operator type and add a message.
 - If the node is an assignment (Assign), extract the assigned variable and add a message.
4. Return the list of extracted dependencies.

- **Pseudocode:**

Algorithm Tokenize_Python_Code(code):

Define tokens_list as an empty list

Define punctuators as a set of special characters

Convert code into a stream using BytesIO

For each token in tokenize(code):

 If token is an encoding marker:

 Continue

 If token is a keyword:

 Mark as "Keyword"

 Else If token is an identifier:

 Mark as "Identifier"

Else If token is a number:

Mark as "Literal (Numeric)"

Else If token is a string:

Mark as "Literal (String)"

Else If token is True or False:

Mark as "Literal (Boolean)"

Else If token is a punctuator:

Mark as "Punctuator"

Else If token is an operator:

Mark as "Operator"

Else:

Mark as "Other"

Add (token, type) to tokens_list

Return tokens_list

Algorithm Extract_Syntactic_Dependencies(code):

Try:

Parse code into an Abstract Syntax Tree (AST)

Define dependencies_list as an empty list

For each node in AST:

If node is a function definition:

Add "[Function Definition] - Function 'name' is defined."

Else If node is a function call:

Add "[Function Call] - 'function_name' is called."

Else If node is a binary operation:

Add "[Binary Operation] - Operator used: operator_type."

Else If node is an assignment:

Add "[Assignment] - Variable 'var_name' is assigned a value."

Return dependencies_list

Catch SyntaxError:

Return "[Syntax Error] - Error Message"

6) User Interaction and Interface Design (Diagram/UML/Architecture):

The system will provide a user-friendly interface for developers, educators, and students to analyze Python code:

Diagram

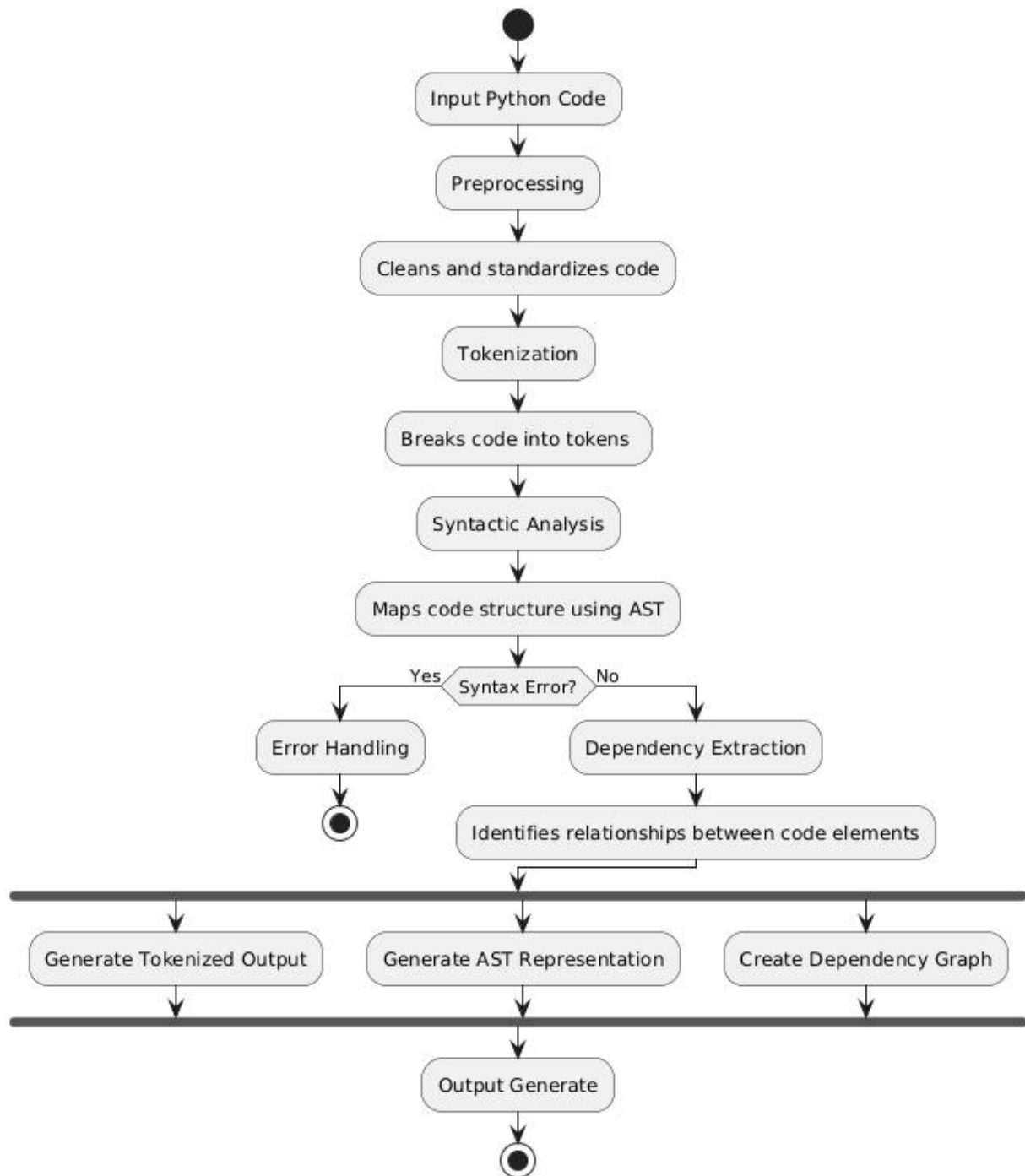


Fig No 1: Activity Diagram

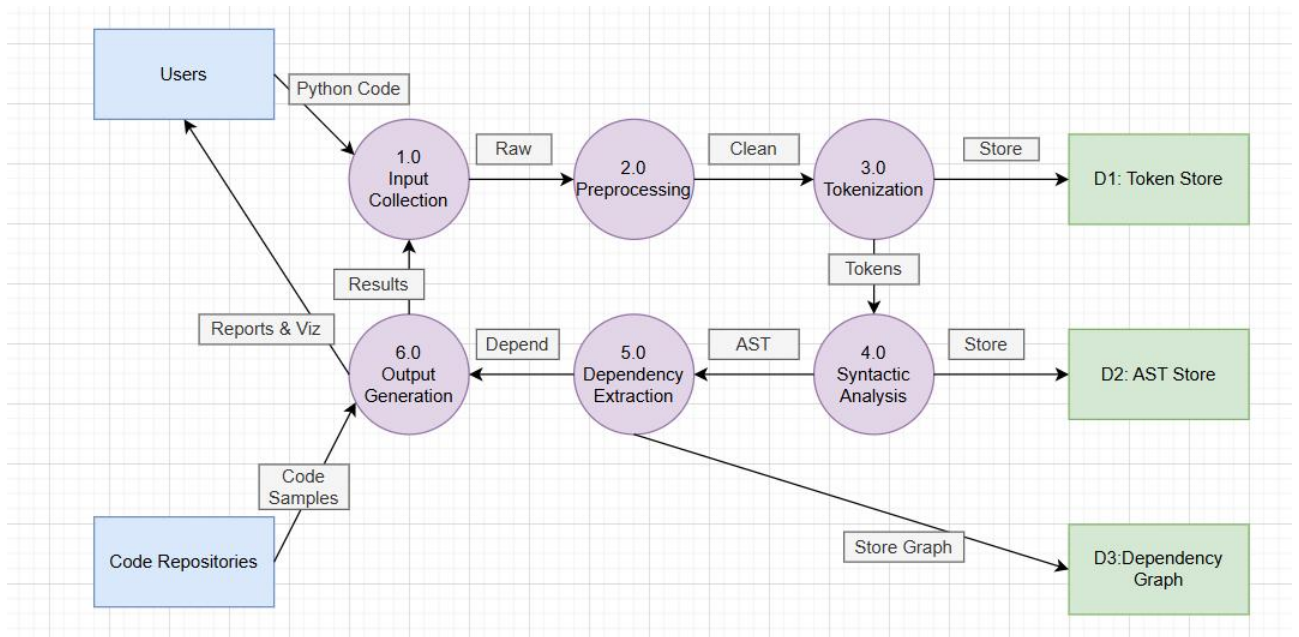


Fig No 2: Data Flow Diagram

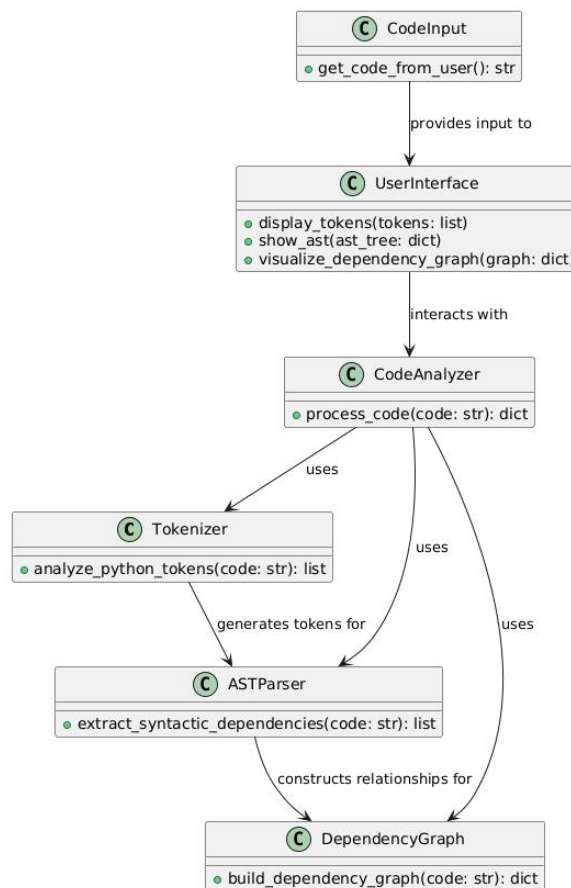


Fig No 3: Class Diagram

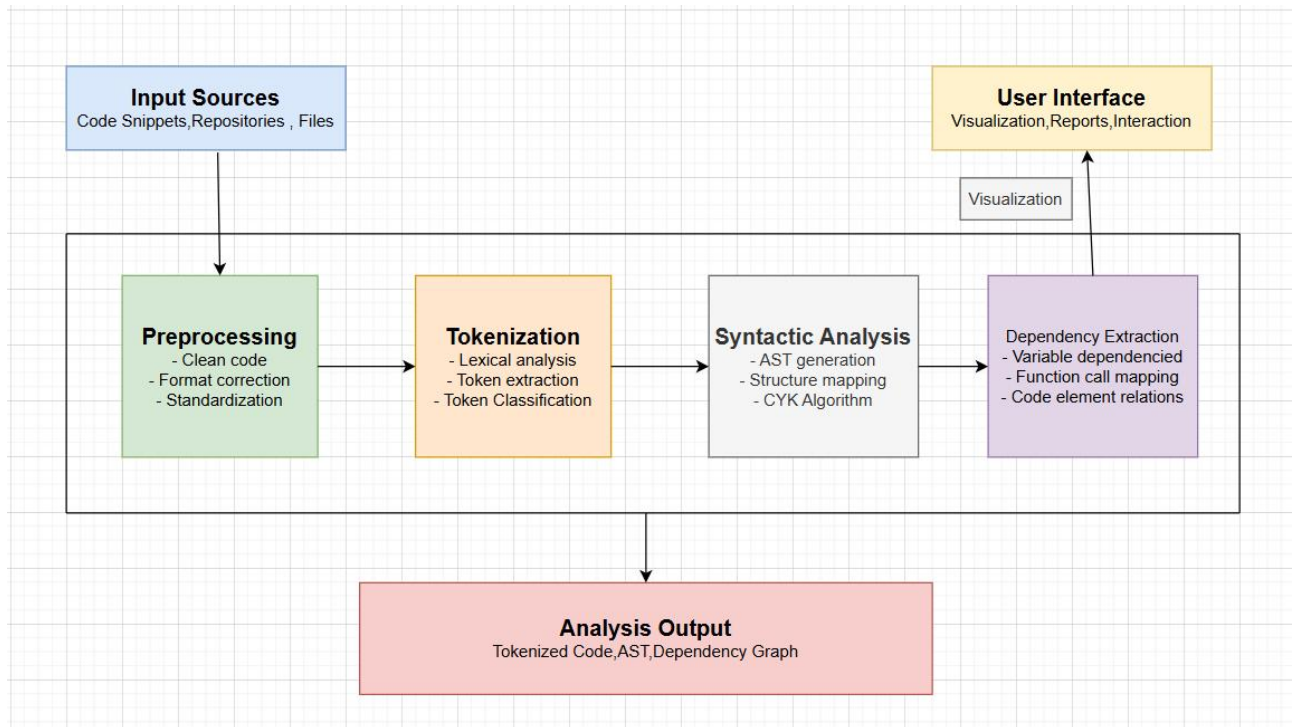


Fig No 4: Architecture Diagram

User Interface Components:

1. **Code Input Panel:** Allows users to paste code snippets or provide URLs to repositories.
2. **Tokenization View:** Shows the tokenized version of the code with color-coding for different token types.
3. **Syntax Tree Visualizer:** Interactive visualization of the AST with expandable/collapsible nodes.
4. **Dependency Graph:** Interactive graph showing relationships between code elements, with filtering options.
5. **Analysis Results Panel:** Displays metrics, potential issues, and insights from the syntactic analysis.
6. **Export Options:** Functionality to export analysis results in various formats (JSON, XML, PDF reports).

7) Testing and Validation Strategy (if applicable)

The testing strategy will ensure the system accurately analyzes Python code across different complexity levels:

1. Unit Testing:

1. Test individual components (tokenizer, parser, dependency extractor)
2. Verify correct handling of specific Python constructs
3. Ensure proper error handling for malformed code

2. Integration Testing:

1. Test the entire pipeline from code input to dependency graph generation
2. Verify correct interactions between modules

3. Validation Dataset:

1. Create a curated dataset of Python code snippets with manually annotated syntactic structures
2. Include examples of varying complexity and covering diverse Python features
3. Measure tokenization accuracy, AST correctness, and dependency identification precision

4. Cross-Validation:

1. Compare results with existing Python parsing tools (ast module, pylint, etc.)
2. Identify discrepancies and refine algorithms

5. User Testing:

1. Conduct sessions with target users (developers, educators, students)
2. Gather feedback on interface usability and analysis accuracy

8)Expected Output

The system will produce comprehensive syntactic analyses of Python code snippets, including:

1. **Tokenized Code:** Complete breakdown of code into tokens with metadata
2. **Abstract Syntax Tree:** Hierarchical representation of code structure, exportable as JSON or visualization

3. **Dependency Graph:** Visual and data representation of relationships between code elements
4. **Syntactic Metrics:** Quantitative measures of code complexity, nesting depth, dependency counts, etc.

For example:

```
def add(a, b):  
  
    result = a + b  
  
    return result  
  
x = add(5, 10)
```

This Python Code Syntactic Analysis System will serve as a valuable tool for code understanding, education, and quality improvement. By accurately mapping the syntactic structure and dependencies within Python code, it provides insights that can help developers write more maintainable code, assist educators in teaching Python concepts, and guide students in understanding code structure. The system's modular design also allows for future extensions to support additional analysis features or other programming languages. Summarize the key points covered in the design phase report. Emphasize the potential benefits of the proposed system and its alignment with the organization's goals and objectives.

